

Day 1: Modeling the Convoy Problem

Getting ships from homeports to combat zones is an important problem to understand and solve in warfare, both supplying our ships to the fight, and limiting adversarial ships from entering the fight. In fact, this problem was formulated in the mid-1950s by US Air Force researchers during the Cold War for strategic bombing targets of USSR railways. We can similarly build a naval version of this problem. We must escort a logistics convoy from a friendly port (call it s) to a forward operating base (t). The intervening waters contain several chokepoints: straits, sea lanes, and contested transit zones. Intelligence has estimated, for each transit leg between waypoints, both a *risk cost* and a *capacity* (the maximum number of ships that can transit that leg per day before sensors and shore-based threats overwhelm the convoy's defenses).

Your job, as the operations planner, is to push as many ships per day from s to t as the network will allow, and then to convince the COCOM that no plan can do better.

1.1 First instinct: shortest path

The natural starting point (one we already have a tool for) is to ask: *what is the safest single route from s to t ?* If we treat the risk cost on each leg as an edge weight, this is exactly a single-source shortest-path problem on a weighted graph. From Unit 5 we know an algorithm to solve this:

- **Dijkstra's algorithm** in $O((|V| + |E|) \log |V|)$ when all edge weights are non-negative;

So in principle, we are done! Pick the minimum-risk path and send the convoy along it. Right?

Common Mistake: A single shortest path tells you the best route for *one* ship. It does not tell you how to move *many* ships, especially when each transit leg can only safely carry a limited number of ships per day. The shortest-path answer collapses as soon as the optimal route saturates.

1.2 Why the model is wrong

Suppose the safest path from s to t runs through a particular strait that can sustain at most 2 ships per day. If we have 6 ships to move, we cannot simply send all 6 down that path on day one; the strait will not absorb them. We need to *split* the convoy across multiple routes, some of which are individually riskier but together carry more total ships.

This is the central modeling shift of the unit: we move from *find the best single route* to *find the best simultaneous routing of many units through a capacitated network*. The mathematical object we need is a **flow network**.

1.3 Flow networks: definitions

A **flow network** is a directed graph $G = (V, E)$ in which:

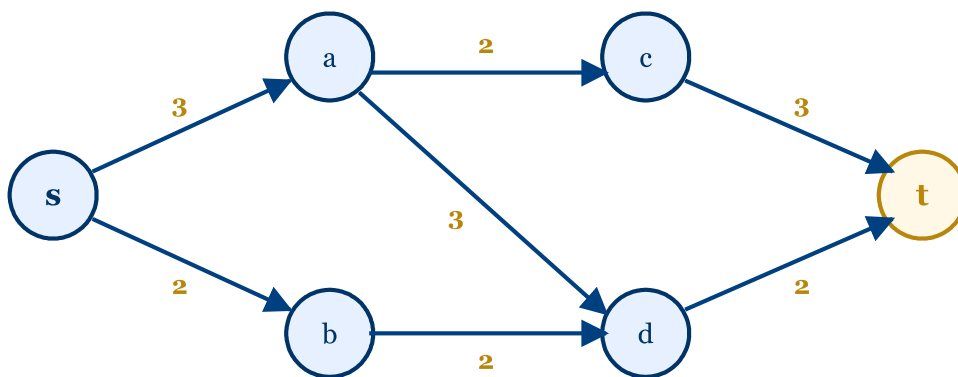
- Two vertices are distinguished: a **source** $s \in V$ and a **sink** $t \in V$.
- Every edge $(u, v) \in E$ has a non-negative **capacity** $c(u, v) \geq 0$.
- If $(u, v) \notin E$, we set $c(u, v) = 0$ by convention.

A **flow** in G is a function $f : V \times V \rightarrow \mathbb{R}$ satisfying:

1. **Capacity:** $0 \leq f(u, v) \leq c(u, v)$ for every edge.
2. **Conservation:** for every vertex $v \neq s, t$, $\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w)$.
Whatever flows in to v must flow out of v .

The **value** of the flow is $|f| = \sum_v f(s, v) - \sum_v f(v, s)$, the net flow leaving the source.

The **maximum flow problem** asks: given a flow network, find a flow f of the largest possible value.



A small flow network. Edge labels are capacities (in ships/day). What is the maximum number of ships per day we can route from s to t ?

1.4 Why we cannot just be greedy

A natural first attempt: repeatedly find *any* s -to- t path with spare capacity, push as much as possible along it, and repeat until no such path exists. This is essentially a greedy algorithm, and it captures the right idea ... but it can get *stuck*.

Consider a network where the first greedy choice saturates an edge that, in the optimum solution, should have carried flow in the opposite direction. Once we commit, the greedy procedure cannot back out. This is exactly the kind of situation where greedy reasoning fails (recall Unit 3): a locally optimal choice forecloses globally optimal options.

The fix, which we will develop later, is to allow the algorithm to *cancel* previous flow decisions by sending flow along a "virtual" reverse edge. That single idea is what makes Ford–Fulkerson work, and it is why a problem that *looks* greedy is actually deeper.

Check Your Understanding: Day 1

1. Why does treating the convoy problem as a single shortest-path problem give the wrong answer?
2. State the two conditions a function f must satisfy to be called a flow.
3. In the network of Figure above, explain in one sentence why we cannot achieve a flow of value 6.
4. Give an example (in words is fine) of a situation where a "send flow down the best available path" greedy algorithm could lock in a suboptimal solution.

Day 2: Maximum Flow and Ford–Fulkerson

2.1 The key idea: residual graphs

To make a path-augmenting algorithm work, we need to give it the ability to *undo* previous decisions. Ford and Fulkerson's idea (1956) is to maintain a separate graph (the **residual graph**) that tracks not just remaining capacity but also the option to reverse previously committed flow.

Given a flow network G and a flow f , the **residual graph** G_f has the same vertices as G , and for every edge (u, v) in G :

- A **forward residual edge** (u, v) with capacity $c(u, v) - f(u, v) \rightarrow$ the unused capacity, if any.

- A **backward residual edge** (v, u) with capacity $f(u, v) \rightarrow$ representing the option to cancel that much flow.

Edges with zero residual capacity are simply omitted.

An **augmenting path** is any directed path from s to t in the residual graph G_f . Its **bottleneck** is the smallest residual capacity along the path.

If an augmenting path exists, we can push *bottleneck* additional units of flow from s to t by adding capacity along forward edges, and subtracting flow along reverse edges (which corresponds to "rerouting" earlier decisions). The Ford–Fulkerson method is simply: repeat until no augmenting path remains.

2.2 The algorithm

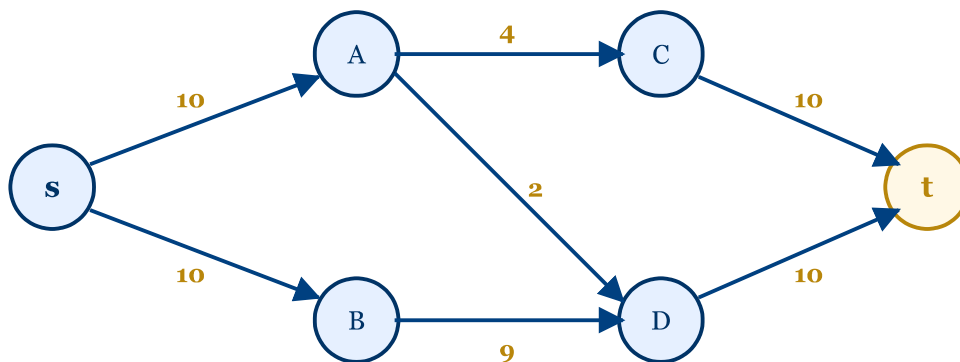
```
function Ford-Fulkerson(G, s, t):
    # Initialize all flows to zero
    for each edge (u,v) in G:
        f(u,v) ← 0

    while there exists an augmenting path P from s to t in G_f:
        b ← bottleneck capacity of P
        for each edge (u,v) in P:
            if (u,v) is a forward residual edge:
                f(u,v) ← f(u,v) + b
            else:
                f(v,u) ← f(v,u) - b          # reverse edge to cancel flow
        return f
```

Notice what is left vague: *how* do we find the augmenting path? Ford and Fulkerson did not specify. Different choices give very different running times. The most important practical choice is to use BFS, which gives the variant called **Edmonds–Karp**. We will see why BFS specifically later.

2.3 Worked example

Let us trace the algorithm on the following small network. Capacities are written on each edge.



A six-node example

Iteration 1. BFS finds the path $s \rightarrow A \rightarrow C \rightarrow t$. Bottleneck = $\min(10, 4, 10) = 4$. Push 4 units. Total flow so far: 4.

Iteration 2. BFS finds the path $s \rightarrow B \rightarrow D \rightarrow t$. Bottleneck = $\min(10, 9, 10) = 9$. Push 9 units. Total flow so far: 13.

Iteration 3. BFS finds the path $s \rightarrow A \rightarrow D \rightarrow t$. Bottleneck = $\min(6, 2, 1) = 1$. Push 1 unit. Total flow so far: 14.

Iteration 4. No augmenting path remains. The maximum flow is $|f^*| = 14$.

Notice that in iteration 3 we used the edge $A \rightarrow D$ to "rebalance" the flow. Without that option, we would have stopped at 13. This is where the residual graph earned its keep.

2.4 Why BFS? Edmonds–Karp and the polynomial bound

The original Ford–Fulkerson method only guarantees termination when capacities are integers; with arbitrary real capacities, it may not terminate at all, and even with integer capacities the number of iterations can be as large as the maximum flow value $|f^*|$, which can be exponential in the input size.

Edmonds and Karp (1972) showed that if we always pick the augmenting path with the *fewest edges* (i.e., a BFS shortest path in the residual graph), the number of augmentations is bounded by $O(|V| \cdot |E|)$, regardless of capacities. Each BFS takes $O(|E|)$ time, so the total running time is:

Edmonds–Karp Running Time

$$T(|V|, |E|) = O(|V| \cdot |E|^2)$$

This is polynomial in the size of the input graph and *independent of the capacity values*. This is critical when capacities can be large.

Why this matters! Notice that Edmonds–Karp’s improvement over plain Ford–Fulkerson is purely a matter of *analysis* ... The algorithm is the same, just with a smarter rule for picking paths. This is exactly the kind of refinement this class is all about: sharper bound, totally different practical guarantees.

Check Your Understanding: Day 2

1. What is the residual capacity of a backward residual edge (v, u) , and what does it represent operationally?
2. In the worked example, redraw the residual graph after iteration 2 (you only need the edges incident to A, C, D).
3. If every capacity in a flow network were doubled, would Edmonds–Karp’s running time bound change? Why or why not?
4. True or false: the flow returned by Ford–Fulkerson is unique. Justify briefly.

Day 3: Min-Cut, Duality, and the Interdictor’s Problem

3.1 The defender’s question, the attacker’s question

So far we have been answering the planner’s question: *how many ships per day can I move?* Let us now switch sides. Imagine you are the adversary’s planner. You cannot stop ships once they sail, but you can *cut* sea lanes: sow mines, deploy submarines and destroyers, establish blockades. Each leg you sever has a cost (in resources, in escalation risk). Your question is: *which set of legs, if I cut them, would most reduce the convoy’s throughput?*

It turns out (and this is really cool!) the defender’s optimum and the attacker’s optimum are *equal*. The maximum flow you can push through the network and the minimum capacity you must remove to cut s from t are the same number. This is the Max-Flow Min-Cut Theorem.

3.2 Cuts, formally

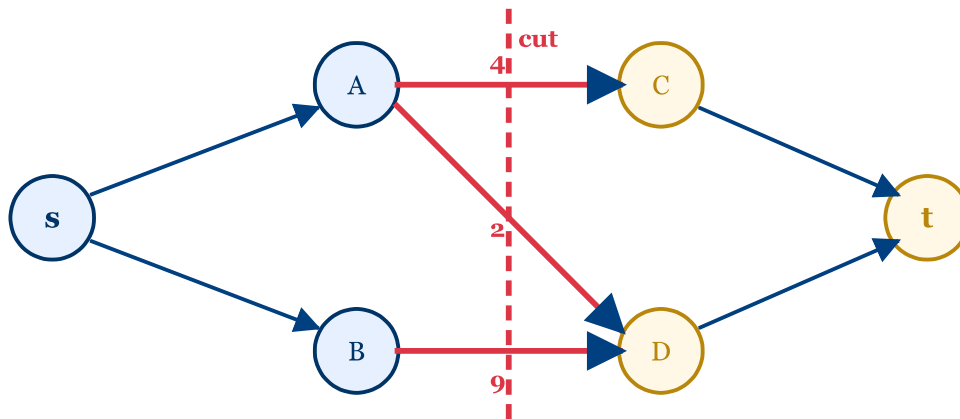
An s - t **cut** of a flow network is a partition of the vertices V into two sets S and $T = V \setminus S$ with $s \in S$ and $t \in T$.

The **capacity** of the cut (S, T) is

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v),$$

the total capacity of edges crossing from s to t . (Edges crossing from t back to s do *not* count.)

Intuitively, a cut is a way of “drawing a line” across the network so that s is on one side and t is on the other. The cut’s capacity is the total capacity of the edges the line crosses.



The cut $S = \{s, A, B\}$, $T = \{C, D, t\}$ in the network in 2.3. Three edges cross the cut, with total capacity $4+2+9=15$. So the max flow is at most 15. Can we find a cut with smaller capacity?

3.3 Max-Flow Min-Cut Theorem

Max-Flow Min-Cut Theorem (Ford & Fulkerson, 1956)

For any flow network, the value of the maximum s - t flow equals the capacity of the minimum s - t cut:

$$\max_f |f| = \min_{(S, T)} c(S, T).$$

Two halves are easy to see and one half is the whole game:

Sketch: Why max-flow $\leq$ min-cut (the easy direction)

Take *any* flow f and *any* cut (S, T) . Every unit of flow that ultimately reaches t must, at some point, cross the cut from s to t . Therefore $|f| \leq c(S, T)$. Since this holds for any flow and any cut, in particular it holds for the maximum flow and the minimum cut.

So we get the inequality $\max_f |f| \leq \min_{(S, T)} c(S, T)$ for free. This is sometimes called *weak duality*.

Sketch: Why max-flow $\geq$ min-cut (the harder direction)

Suppose Ford–Fulkerson terminates with flow f . Then by definition there is no augmenting path in G_f . Define

$$S = \{v \in V : V \text{ is reachable from } s \text{ in } G_f\}, \quad T = V \setminus S.$$

Clearly $s \in S$, and $t \in T$ since no augmenting path exists. So (S, T) is a valid s - t cut.

Now consider any edge (u, v) from s to t in the original graph. If it had any residual capacity left, then V would be reachable from s in G_f , contradicting $v \in T$. So $f(u, v) = c(u, v)$: every forward edge across the cut is saturated. Similarly, every edge from t to s has $f = 0$ (otherwise its reverse would appear in G_f). Adding it up, $|f| = c(S, T)$.

So we have exhibited a flow and a cut with equal value, which (combined with weak duality) proves they must both be optimal.

The proof does more than equate two numbers. It tells us that *Ford–Fulkerson, when it halts, simultaneously hands you both an optimal flow and a witness cut*. The cut is the certificate that no better flow is possible. Operationally: the algorithm gives the planner a route plan *and* a one-line argument the commander can verify.

3.4 Reframing as the interdicator's problem

The min-cut perspective is exactly the interdicator's problem in disguise! At least, the version where the interdicator pays a cost equal to each edge's capacity to remove it and wants to minimize total cost while reducing the throughput to zero.

In other words, max-flow and min-cut are **dual**: solving one is equivalent to solving the other. From the planner's chair, max-flow gives you the best plan. From the adversary's chair, min-cut gives the cheapest blockade. They are the same number.

Why this matters! Recall that BFS, which we used in Edmonds–Karp, also gave us shortest paths in unweighted graphs. The same data-structure-level workhorse is now solving a problem about *capacities*. This is a recurring theme: a small set of primitives (BFS, DFS, sorting, recursion) gets reused across very different problem types.

Check Your Understanding: Day 3

1. In the cut figure, what is the capacity of the cut $S = \{s\}$, $T = \{A, B, C, D, t\}$? Is this the minimum cut?
2. Explain in plain language why the max-flow value cannot exceed any cut capacity.
3. Suppose Edmonds–Karp returns a flow of value 14 on a network. What additional information does the algorithm give you, essentially for free, that would convince a skeptic the flow is optimal?
4. If an interdicator cared not about edge capacities but about the *number* of edges removed (a unit-cost model), would the min-cut answer the same question? Why or why not?

Day 4: When the Interdicator Has a Budget and Hardness

4.1 Where the duality breaks

Last class left us in a comfortable place: routing and interdiction were dual, both polynomial, both solved by the same algorithm. So one might reasonably expect that *any* reasonable variation of the interdiction problem is also polynomial. It is not.

The min-cut problem charges the interdictor in proportion to each edge's capacity. But realistic adversaries face a different constraint: a *budget* on how many edges they can attack. Mining an additional sea lane is mining an additional sea lane so its cost in assets and political capital is roughly the same regardless of how much shipping that lane normally carries.

k -Edge Network Interdiction (Decision Version).

Input: Flow network $G = (V, E)$ with source s , sink t , integer capacities, integer budget k , and target value t .

Output: Is there a set $E' \subseteq E$ with $|E'| \leq k$ such that the maximum flow in $G - E'$ is at most t ?

The key difference from min-cut: the interdictor can remove *at most k edges*, regardless of their capacities, and wants to push throughput below some target. This innocent-looking change destroys the polynomial structure.

4.2 Why this is hard

Theorem

k -Edge Network Interdiction is **NP-complete**.

We will not give the full proof. The standard one involves a reduction from the Clique problem (which you did in your FEX, and is NP-Complete!) and is intricate, but the intuition is worth understanding.

In the polynomial min-cut problem, the algorithm exploits a global property: the optimum cut is *structural*, determined by the topology and the residual graph. Once you constrain the interdictor to a fixed *count* of edges rather than a sum of capacities, the problem becomes combinatorial in a fundamentally different way: which k -subset of edges achieves the most damage? There are $\binom{|E|}{k}$ such subsets, and no greedy or LP-based approach is known to find the right one in polynomial time.

Loosely, the reduction works like this: a hard combinatorial problem (Clique, Independent Set, or 3-SAT depending on the construction) can be encoded as a network where the only way to push throughput below the target with k edge removals is to "select" a structure that corresponds to a solution of the hard problem. If interdiction were polynomial, so would the hard problem be, contradicting our complexity assumptions from Unit 6.

Common Mistake: The lesson here is general and important. *Polynomial-time solvability is fragile.* A small, operationally reasonable change in the problem (paying per edge rather than per unit of capacity) can flip a problem from “solvable in seconds” to “no known efficient algorithm.” Recognizing when this happens is one of the most useful skills the course can give you.

4.3 What planners actually do

NP-hardness is not the end of the conversation. Real defense analysts and operations researchers solve interdiction problems every day. They do it by combining several tools you have already seen:

1. **Mixed-Integer Linear Programming (MILP).** The problem can be written as an integer program and handed to a solver like CPLEX or Gurobi. These solvers use branch-and-bound (which we touched on in Unit 6) and clever LP relaxations. They are exponential in the worst case but extremely fast on most realistic instances.
2. **LP relaxation as a lower bound.** Drop the integer constraint, solve the resulting LP in polynomial time, and use its objective as a bound. If the LP optimum is already above the threshold, the integer optimum is too. This is the same logic as branch-and-bound pruning.
3. **Approximation algorithms.** For some interdiction variants, polynomial-time algorithms exist that guarantee a solution within a known constant factor of the optimum. Trading exactness for tractability is exactly the move we discussed at the end of Unit 6.
4. **Heuristics with verification.** Run a fast greedy or local-search heuristic to find a solution, then use the LP relaxation or a partial computation to bound how far it could be from optimal. If the gap is small enough for the decision at hand, ship it.

None of these techniques would have made sense to us in the beginning of the semester. By now, hopefully some this seems familiar.

4.4 The story of the semester, told through this case study

Step back. Look at what the routing-and-interdiction case study has used:

- **Unit 1 (Analysis).** The whole reason we care that Edmonds–Karp is $O(|V| \cdot |E|^2)$ rather than “depends on capacity values” is that we know how to read and trust an asymptotic bound. The same bound on plain Ford–Fulkerson would have looked acceptable until you tried it on a realistic problem.
- **Unit 2 (Divide and Conquer).** Not directly used in the flow algorithms, but the recursive structure of branch-and-bound search trees in Section 4.3 is exactly the divide-and-conquer pattern, applied to a search space rather than an array.